UNIVERSITY OF COPENHAGEN
FACULTY OF SOCIAL SCIENCES
DEPARTMENT OF ECONOMICS

Master Thesis

Tuan Anh Nguyen

Mid-Price prediction with Limit Order Book data and the Efficient Market Hypothesis

Advisor: ————
Submission date: 01.11.2024
Character counts: ≈ 54200

ABSTRACT

The purpose of this thesis is to predict future mid-price movements of High-Frequency Limit Order Book (LOB) data via two successful machine learning algorithms in the literature. The Random Forest and the Support Vector Machine algorithms. This includes extraction of features and applying widely used technical indicators from the LOB data in training the models through cross-validation and hyper-parameter tuning. The optimal model are then used to formulate and simulate a model-based trading strategy and evaluate whether it outperforms the benchmark buy-and-hold trading strategy, and thereby challenging the Efficient Market Hypothesis. Our findings did outperform the benchmark strategy when transactions costs was excluded, but the model falls short when transaction costs was incorporated.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1

INTRODUCTION

Forecasting and predicting are crucial activities across various fields, including medicine, weather, and especially financial markets. Predicting future movement in equities and indices allows investors and traders to anticipate market trends more effectively and make informed investment decisions. This is particularly appealing because many people have an inherent desire to seek opportunities for financial gain, whether to improve their financial situation, build wealth, or even proof that they can "*beat*" the market. As a result, many turn to trading as means to achieve such goals.
Trading has evolved significantly from the open outcry systems used on trading floors, where traders communicated buy and sell orders verbally before the 1970s. Since then, electronic trading systems have been widely adopted by many exchanges. Advances in technology and computational power have reduced order processing times, leading to the rise of high-volume, high-frequency trading. Traders use high-frequency trading (HFT) systems to develop strategies that executes trades within milliseconds. The goal is to profit from price discrepancies from market fluctuations. But predicting future price movements and formulating strategies in HFT setting is very challenging due to the high volume of data and the speed of execution.
Traditional finance theory is based on the idea that financial markets are efficient - The Efficient Market Hypothesis (EMH). According to Fama (1970)[1] all publicly available information is immediately incorporated into asset prices and that no excess returns can be achieved through any trading strategy. Because, any potential profit opportunity will be captured by other investors and thus quickly be eroded by market forces. Price movements or price patterns from historical prices cannot be predicted to form strategies that beat the market in the long-term.
The goal of this thesis is first to predict mid-prices, derived from bid and ask prices in the Limit Order Book (LOB) data. Unlike single closing prices, LOB data offers in-depth information about market activities, that can be extracted into features for prediction. Secondly, it seeks to formulate a trading strategy and evaluate whether it outperforms the benchmark buy-anh-hold, thereby challenging the efficiency of the Efficient Market Hypothesis.

Our trading strategy did outperform the buy-and-hold strategy for the long-term of eight years for the data spanning from 2016-2023

without transaction costs. Including transactions, the model did not find evidence against the EMH.

This thesis is organized in the following way. In the following a literature review will be given. The next section describes the genereal Limit Order Book structure from LOBSTER, while section 3 describe the Random Forest and Support Vector Machine algorithms. Section 4 gives an explanation of our methodology. Section 5 present the results from cross-validation and trading profit and loss. In section 6 an brief discussion and conclusion is given.

1.1 LITERATURE REVIEW

The intersection of technology and finance is reshaping the landscape of financial markets Addy *et* al. (2014) [2]. High speed trading execution has impacted market liquidity, volatility and market efficiency. Moreover, the rise of machine learning in the last decades has changed the way we approach prediction of future stock movements. Unlike traditional models that rely heavily on linear assumptions, machine learning models can handle non-linear relationship and interactions between variables, which are a more alike the behavior in stock returns. In ML there are many algorithms and they can be categorized into different branches of learning. The question for the researcher is which learning model should be used to conduct the analysis. For example, Hoisenzade & Haratizadeh (2019) [3] demonstrated how Convolutional Neural Networks (CNN), a deep learning, method could be applied to stock price prediction. They found that CNNs outperformed traditional models, especially when dealing with high-dimensional data. Another example, besides deep learning Henrique *et* al. (2019) [4] used the Random Forests and other ensemble techniques for asset movement prediction. Their findings indicated that these models, combined with feature selection could empower the predictive accuracy.

The limit order book (LOB) data offers detailed view of the microstructure of the market, making the purpose of predicting mid-price movements appealing. In high-frequency trading, a slightest edge can be the difference between profit and loss. Zhang *et* al. (2019) [5], Sirignano & Cont (2019) [6] explored the use of Neural Networks to capture the informative patterns in the LOB to predict mid-price movements. Zhang (2022) [7], Grudniwiecz (2023) [8],Han *et* al. (2015) [9], Nousi *et* al. (2019) [10], Chang (2021) [11], Ntakaris *et* al. (2018) [12], Kercheval & Zhang (2015)[13], and Mangat *et* al. (2022) [14] demonstrated approaches of using the Support Vector Machine (SVM) algorithm, Ridge regression, and Neural Networks in prediction of mid-price of LOB data. Ntakaris *et* al. (2018) refers to their approach as the benchmark method for forecasting of LOB data. As my goal is to predict the mid-price of LOB data, I will use the same algorithm as

Kercheval & Zhang (2015)[13] and Mangat *et* al. (2022)[14], that is, the Support Vector Machine algorithm.
Another consideration is whether or not to include technical indicators. See Hiransha *et* al. (2018) [15], Abbasi *et* al. (2020) [16], Frommel (2015)[17], Jin (2022)[18], and Cohen (2015) [19], Cohen (2015) [19] included technical indicators like the Relative Strength Index (RSI) and Moving Average Convergence Divergence (MACD) indicator in their analysis on on various markets. All three authors suggested, that including technical indicators could improve prediction accuracy. Cohen (2015)[19] found that the inclusion of MACD and RSI contributed to the outperformance on the Nikkei 225 (NK225) index.

Before the success of these machine learning methods in forecasting stock prices, econometric models were the main approaches. These models, based on statistical theory, have been extensively used to predict stock prices and market volatility. An example is, the Autoregressive Conditional Heteroscedasticity (ARCH) model introduced by Engle (1982) [20], and its extension later on, the GARCH model. His models has been central in modeling and predicting volatility in financial markets. But from economic theory, we know these model often assume linear relationship and stationary processes, which can be limiting in the environment of financial markets. Fama & French (1993) [21] predicted stock return with a three-factor model, the model relies on historical averages, thus it limits the ability to adapt to real-time market conditions.
In the literature there are many examples of ML perform better for the task of predicting future prices, one is Gu *et* al (2018) [22]. They identified best performing methods are based on decision trees and neutral networks non-linear settings. Another is Campisi *et* al. (2024) [23], they found that random forest and bagging attain the highest predictive direction of the US stock market on the basis of volatility indices comparable to the classical least squares regression model. These findings support the choice of the Random Forest in this thesis. Also, the Random Forest model is widely used in machine learning due to its ease of implementation and straightforward interpretability. When combined with the more complex SVM, which excels in non-linear settings, these two chosen algorithm complement each other in this thesis for predicting the direction of mid-prices.

2 LIMIT ORDER BOOK

2.1 LIMIT ORDER BOOK

In this section, we describe the structure of high-frequency limit order book data from LOBSTER, where our data is retrieved.

A Limit Order Book (LOB) is an electronic system that records all trading activities of limit orders. A limit order is an order to either buy or sell a stock at a specified price. This means, the LOB has two sides: the bid side and the ask side. The bid side refers to the highest price that a buyer is willing to pay for a stock, while the ask price refers to the lowest price that a seller is willing to sell a stock. In contrast, a market order is an order to buy or sell a stock immediately at the current market price, without specifying a price limit.

The LOB data used in this thesis are from LOBSTER, which is an online limit order book data tool which provides high-quality limit order book data.[1] The limit order book data that LOBSTER reconstructs originates from NASDAQ's Historical TotalView-ITCH.[2] To retrieve the data from LOBSTER, I downloaded the data for the stock of interest by defining the relevant time period. Afterward, LOBSTER generates a *message book* file and an *order book* file for each trading day of the selected stock. The message book file contains information for the type of event causing an update of the limit order book in the requested price range. A sample of the message book file from LOBSTER is displayed below. Following a list of variable explanation of the message book file.

Time (sec)	Event Type	Order ID	Size	Price	Direction
⋮	⋮	⋮	⋮	⋮	⋮
34713.685155243	1	206833312	100	118600	-1
34714.133632201	3	206833312	100	118600	-1
⋮	⋮	⋮	⋮	⋮	⋮

Figure 1: Sample of message book file from LOBSTER. It contains information about each event.

1 https://lobsterdata.com/index.php
2 https://nasdaqtrader.com/Trader.aspx?id=ITCH

- Time: Seconds after midnight with decimal precision of at least milleseconds and up to nanoseconds depending on the period requested
- Event Type:
  1. Submission of a new limit order
  2. Cancellation (partial deletion of a limit order)
  3. Deletion (total deletion of a limit order)
  4. Execution of a visible limit order
  5. Execution of a hidden limit order
  6. Indicates a cross trade, e.g., auction trade
  7. Trading halt indicator[3]
- Order ID: Unique order reference number
- Size: Number of shares
- Price: Dollar price times 10.000 (i.e., a stock price of $99.14 is given by 991400)
- Direction:
  - -1: Sell limit order
  - 1: Buy limit order
  - Note: Execution of a sell (buy) limit order corresponds to a buyer (seller) initiated trade, i.e., buy (sell) trade.

The corresponding sample of the order book file is displayed below. It contains unexecuted limit orders for both sides at a given price level. For each given price level e.g., level one, the order file provides four values - ask price 1, ask volume 1, bid price 1, bid volume 1. This corresponds to the best ask price, best ask volume, best bid price, best bid volume. For the price level 2 we get the corresponding four values - second best ask price, second best ask volume, second best bid price, second best bid volume. This goes on until price level 50 in LOBSTER. As we can see from the the two samples, a change in the message book causes a change in the order book. The $k-th$ row in the message book causing the change from line $k-1$ to line k in the order book. Consider the samples of the message book and order book. The limit order deletion (event type 3) in the second line of the message book removes 100 shares from the ask side at price 118600. The change in the order book from line one to two corresponds to this removal of liquidity. The volume available at the best ask price of 118600 drops from 9484 to 9384 shares.

3 see appendix

Ask Price 1	Ask Size 1	Bid Price 1	Bid Size 1	Ask Price 2	Ask Size 2	Bid Price 2	Bid Size 2	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1186600	9484	118500	8800	118700	22700	118400	14930	...
1186600	9384	118500	8800	118700	22700	118400	14930	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Figure 2: Sample of corresponding order book file from LOBSTER at price level 2

- Ask Price 1: Level 1 ask price (best ask price)
- Ask Size 1 : Level 1 ask volume (best ask volume)
- Bid Price 1: Level 1 bid price (best bid price)
- Bid Size 1: Level 1 bid volume (best bid volume)
- Ask Price 2: Level 2 ask price (second best ask price)
- ...

3

MACHINE LEARNING, THE RANDOM FOREST CLASSIFIER AND THE SUPPORT VECTOR MACHINE

3.1 MACHINE LEARNING

Machine learning is a field of artificial intelligence that focuses on developing algorithms and models that enable computers to learn patterns and make decisions or predictions based on data. Instead of being programmed with a set of rules, machine learning models learn from data, improving their performance when given more examples. There are three main types of machine learning: *supervised learning, unsupervised learning, and reinforcement learning*.
The main goal of supervised learning is to learn a model from labeled training data that allows it to make predictions about unseen or future data. The term *supervised* refers to a set of samples where the desired output signals (labels) are already known.[24] Given input variables, X and output variables, Y the algorithm observes the system for both inputs and outputs, and assembles a *training* set of observation $(x_i, y_i), i = 1, ..., N$ and attempts to produce an estimated function $\hat{f}$. After learning about $\hat{f}$, the algorithm can predict outputs for new inputs that were not a part of the training set.
The second type of machine learning is reinforcement learning. In reinforcement learning, the goal is to develop a system (agent) that improves its performance based on interactions with the environment. Hence, this learning method does not use labeled data, but the agent learns by receiving reward signal or penalties based on its sequence of decisions by interacting with the environment. For instance, the algorithm is playing multiple games of the abstract strategy game *Go* against itself, to learn and improve over time.
The last type of machine learning is unsupervised learning. In supervised learning we know the information about the inputs and outputs beforehand when we train our model. In unsupervised learning, however, we are dealing with unlabeled data and unknown structure of the data. Using this technique, the algorithm is trying to explore the structure of our data to extract meaningful information.

For my analysis, I will be using two popular supervised learning algorithms: the Random Forest Classifier and the Support Vector Machine (SVM). In the following the theories of these classifiers are introduced. As the goal of this thesis is to predict the mid-price direction,

I have chosen these two successful approaches from the literature to obtain the best predictor.

3.2 THE RANDON FOREST CLASSIFIER

To understand the Random Forest Classifier, it is important to first introduce the concept of the Decision Tree Classifiers, as the Random Forest classifier is essentially an ensemble of multiple decision trees.

3.2.1 *The Decision Tree*

A decision tree is a machine learning technique that is used to solve classification and regression problems. Classification is a subcategory of supervised learning where the goal is to sort observations into discrete class labels or categories, based on past observations. For example, an algorithm can learn to predict whether an email is spam or not by analysing words from earlier labeled emails. When a new email arrives, the algorithm will classify the email into two categories spam or non-spam emails based on some specific words it learned from the past. This case is a binary classification with two classes. It can also classify observations into multiple categories like a TV series into different genres like thriller, action, or drama etc. In contrast, regression, the other subcategory of supervised learning, where the goal is to predict a specific numerical value based on observations. For example, an algorithm using data of years of experience and education level, a regression model can estimate a person's salary.
A decision tree consists of a root node, a decision node, and a leaf node. We can think of this tree as breaking down our data by making decisions based on a series of questions with certain threshold values. Each question helps to infer which branch or node each observation belongs to. Starting from the root node of the decision tree, the algorithm iterative splits the data into branches from its nodes until the leaves at each node are pure. In order to split the nodes at the most informative features, we need to define an objective function that the tree learning algorithm aims to optimize. Our objective function is to maximize the Information Gain (IG), which is given by:

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^{m} \frac{N_j}{N_P} I(D_j)$$

where, f is the feature to perform the split, D_p and D_j are the dataset of the parent and jth child node, I is the impurity measure, N_p is the total number of observations at the parent node, and N_j is the number of observations in the jth child node. We can see, the information gain is the difference between the impurity of the parent

node and the sum of the child node impurities. The lower the impurity of the child nodes, the larger the information gain. This means that each parent node is split into two child nodes, D_{left} and D_{right}:

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

Three impurity measures of the splitting that are commonly used in binary decision trees are *Gini impurity* (I_G), *Entropy* (I_H), and *Classification Error* (I_E). In practice, the Gini impurity and entropy often yields similar results whereas the measure of classification error is not recommended for growing a decision tree[1]. The definition of entropy is given by:

$$I_H(t) = -\sum_{i=1}^{c} p(i|t) \log_2 p(i|t), \quad \forall \quad p(i|t) \neq 0$$

where $p(i|t)$ is the being the proportion of the observations that belong to class c for a particular node t. The entropy is 0 if all observations at a node belongs to the same class, and the maximal entropy of 1 when the class is uniform distributed. For instance, in the binary class, the entropy is 0 if $p(i = 1|t) = 1$ or $p(i = 0|t) = 0$ and if $p(i = 1|t) = 0.5$ or $p(i = 0|t) = 0.5$, the entropy is 1. We say that the entropy measure maximizes the mutual information in the tree.
The Gini impurity is given by:

$$I_G(t) = \sum_{i=1}^{c} p(i|t)(1 - p(i|t)) = 1 - \sum_{i=1}^{c} p(i|t)^2$$

and this measure can be understood as criterion to minimize the probability of misclassification. In the Random Forest model I will be using the Gini impurity measure.

3.2.2 *Random Forest*

As mentioned earlier, the Random Forest Classifier can be considered as an ensemble of multiple decision trees. The idea behind a random forest is to average multiple decision trees that individually suffer from high variance, to build a more robust model. This model then has a better generalization performance and is less prone to overfitting. Overfitting is when an algorithm fits too closely to its training data and the model cannot make informative or accurate predictions on new data. This algorithm have gained popularity in applications of machine learning for the last decade due to their good classification performance and ease to use. The random forest algorithm can be summarized in these following steps:

1 See Raschka (2017) [24]

1. Draw a random bootstrap sample of size n (randomly choose n samples from the training set with replacement).
2. Grow a decision tree from the bootstrap sample. At each node:
   - i) Randomly select a set of features without repalcement
   - ii) Split the node using the feature that that provides the best split according to the impurity measure.
3. Repeat the steps of 1-2 k times.
4. Aggregate the prediction by each tree to assign the class label by *majority vote* - getting a vote from each tree and then classifies using majority vote.

A big advantage of random forest is that we do not have to worry about choosing good hyperparameter values. We typically do not need to prune the random forest since the ensemble model is robust to noise from the individual decision trees. The only parameter we care about in practice is choosing the number of trees, k, from step three above for the Random Forest. Typically, the larger the number of decision trees the better the performance but at the expense of an increased computational cost.

3.3 THE SUPPORT VECTOR MACHINE (SVM)

The Support Vector Machine (SVM) is a supervised machine learning model used for classification and regression problems. It is an efficient and powerful algorithm capable of performing both linear and non-linear classification. The main idea is to find a hyperplane that can separate the observations into different classes. The optimal hyperplane is the one that maximizes the margin between the closest points belonging to the different classes.

In cases where complex data is not easily separable by a linear boundary, the Support Vector Machine approaches this with a technique called the kernel trick. This is done by mapping the data into a higher-dimensional space, where it becomes easier to find a separating hyperplane, which will allow for a nonlinear decision boundary in the original space.

Consider a binary classification problem. The training data is given as N pairs $(x_1, y_1), (x_2, y_2), ..., (x_N, y_N)$ where $x_i \in R^p$ are the observations and $y_i \in \{1, -1\}$ are the class labels.

We define a hyperplane:

$$\{x : f(x) = x^T\beta + \beta_0 = 0\}$$

where β is the unit vector $||\beta|| = 1$.

From $f(x)$, we establish a classification rule:

$$G(x) = \text{sign}[x^T\beta + \beta_0]$$

Sometimes data points can not be perfectly separated by a boundary, thus we wish to distinguish between two cases: *the separable case* and *the nonseparable case*.

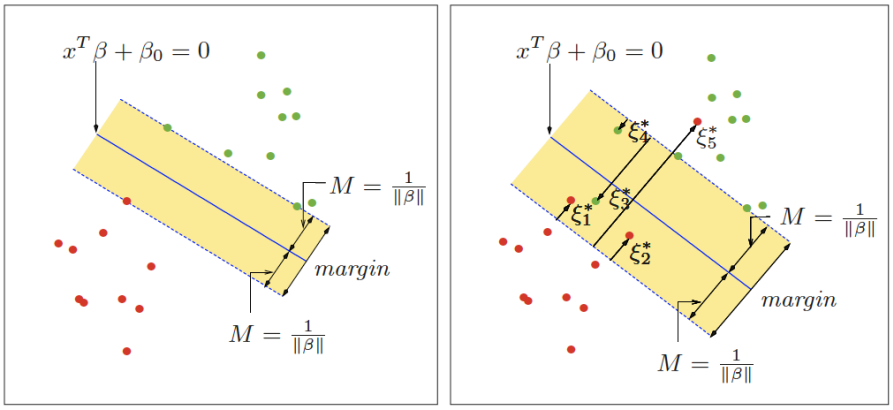

Figure 3: The separable case (left) vs. the nonseparable case (right). In the separable case, clearly, all data points are successfully separated by a solid line. In the nonseparable case, some data points are situated on the wrong side of the margin (Hastie *et* al. (2009)[25]

The margin is the distance from $f(x)$ to the observations from each class that are closest to the boundary. These observations mark the edge of the margin, so they are called edge observations or support vectors. By looking at Figure 3, we see that the margin must have a width of 2M, since each side is $M = \frac{1}{||\beta||}$.

The separable case

If data is separable, then we seek to determine the max-margin hyperplane, which is the hyperplane that separates the classes, class 1 and -1, and allows for the largest possible margin between the edge observations and the hyperplane.

To do this, consider the following optimization problem:

$$\begin{aligned} \max_{\beta,\beta_0,||\beta||=1} \quad & M \\ \text{s.t.} \quad & y_i(x_i^T\beta + \beta_0) \geqslant M, i = 1, ..., N \end{aligned} \tag{1}$$

While this can seem like the most intuitive way to phrase the problem, it is nonetheless a nonconvex optimization problem. To make it a convex optimization problem, rewrite (1) by dropping the norm constraint on β and using that $M = \frac{1}{||\beta||}$. This leads to the following minimization problem:

$$\begin{aligned} \min_{\beta,\beta_0} \quad & \|\beta\| \\ \text{s.t.} \quad & y_i(x_i^T\beta + \beta_0) \geqslant 1, i = 1, ..., N \end{aligned} \tag{2}$$

To solve this problem, define the Lagrange (primal) function as:

$$L_P = \frac{1}{2}\|\beta\|^2 - \sum_{i=1}^{N} \alpha_i[y_i(x^T\beta + \beta_0) - 1] \tag{3}$$

Find the derivatives w.r.t β and β_0 and set them equal to zero. Then we get:

$$\frac{\partial L_P}{\partial \beta} = 0 \Leftrightarrow \sum_{i=1}^{N} \alpha_i y_i x_i = \beta \tag{4}$$

$$\frac{\partial L_P}{\partial \beta_0} = 0 \Leftrightarrow \sum_{i=1}^{N} \alpha_i y_i = 0 \tag{5}$$

s.t. $\alpha_i \geqslant 0 \wedge \sum_{i=1}^{N} \alpha_i y_i = 0$

By substituting these into (2), we obtain the Lagrangian dual objective function:

$$L_D = \sum_{i=1}^{N} \alpha_i - \sum_{i=1}^{N}\sum_{i'=1}^{N} \alpha_i \alpha_{i'} y_i i_{i'} x_i^T x_{i'} \tag{6}$$

To make sure that the Karush-Kuhn-Tucker conditions are satisfied, we need to add:

$$\alpha_i[y_i(x_i^T\beta + \beta_0) - 1] = 0, \forall i \tag{7}$$

Equations (4)-(7) uniquely characterizes the solution to the primal and dual problem.New observations x_j are classified by the optimal hyperplane:

$$\hat{f}(x) = x^T\hat{\beta} + \hat{\beta}_0$$

The nonseparable case

If data is nonseparable, then this means that the classes overlap in the feature space. To account for this in a maximization problem, we will allow some data points, up to a threshold, to be on the wrong side of the margin. This type of margin is called a soft margin, and the aim is to find the optimal soft margin. The concept of a soft margin is illustrated in Figure 3, where the errors of the wrongly classified observations are measured as slack variables $\xi = (\xi_1, \xi_2, ..., \xi_N)$. We

reconstruct the previous optimization problem, by changing the constraint in (2) to:

$$y_i(x_i^T\beta + \beta_0 \geqslant M(1-\xi_i)$$

where $\xi_i \geqslant 0$ and $\sum_{i=1}^{N} \xi_i \leqslant C\ \forall i$. Then the convex minimization problem can conveniently be written as the following:

$$\begin{aligned} \min_{\beta,\beta_0} \quad & \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{N}\xi_i \\ \text{s.t.} \quad & y_i(x_i^T\beta+\beta_0) \geqslant 1-\xi_i, \forall i \\ & \xi_i \geqslant 0 \end{aligned} \tag{8}$$

where the cost parameter C defines the number of misclassifications that are allowed beyond the wrong side of the margin.

The Lagrange (primal) function is:

$$L_P = \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{N}\xi_i - \sum_{i=1}^{N}\alpha_i[y_i(x_i^T\beta+\beta_0)-(1-\xi_i)] - \sum_{i=1}^{N}\mu_i\xi_i \tag{9}$$

which is minimized w.r.t β_0, β and ξ_i. Finding the derivatives and setting them equal to zero gives:

$$\beta = \sum_{i=1}^{N}\alpha_i y_i x_i \tag{10}$$

$$0 = \sum_{i=1}^{N}\alpha_i y_i \tag{11}$$

$$\alpha_i = C - \mu_i, \forall i \tag{12}$$

and constraints $\alpha_i, \mu_i, \xi_i \geqslant 0\ \forall i$. Substitute (10)-(12) into (9) leads us to the Lagrangian dual objective function:

$$L_D = \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{i'=1}^{N}\alpha_i\alpha_{i'}y_i y_{i'}x_i^T x_{i'} \tag{13}$$

To maximize L_D s.t. $0 \leqslant \alpha_i \leqslant C$ and $\sum_{i=1}^{N}\alpha : iy_i = 0$. To satisfy the Karush-Kuhn-Tucker conditions, the following constraints are needed:

$$\alpha_i[y_i(x_i^T\beta + \beta_0) - (1 - \xi_i)] = 0 \tag{14}$$
$$\mu_i\xi = 0 \tag{15}$$
$$y_i(x_i^T\beta + \beta_0) - (1 - \xi_i) \geqslant 0 \tag{16}$$

The equations (10)-(16) gives us a unique solution of the primal and dual problem. Recall from equation (10) that the optimal solution for β is:

$$\hat{\beta} = \sum_{i=1}^{N} \hat{\alpha}_i y_i x_i$$

where only the support vectors have nonzero coefficients $\hat{\alpha}_i$, and $\hat{\beta}$ is only dependent on these.

Given the results of $\hat{\beta}_0$ and $\hat{\beta}$, the final decision function is:

$$\hat{G}(x) = \text{sign}[\hat{f}(x)] \tag{17}$$
$$= \text{sign}[x^T\hat{\beta} + \hat{\beta}_0] \tag{18}$$

Thus, (18) is a linear solution to the nonseparable case. Note that the cost parameter C can be estimated by cross-validation. In some cases, when data is more complex, it can become necessary to look for a nonlinear solution.

The Kernel trick

So far, we have only considered linear boundaries in the input feature space. While this works well on simple data, more complex data may require a nonlinear solution to achieve better training-class separation. By enlarging the feature space with basis expansions, the model becomes more flexible, allowing linear boundaries in the enlarged space to correspond to the nonlinear boundaries in the original space.

Define the m'th input space as $h_m(x_i)$ for $m = 1, ..., M$. While it is not necessary to state the transformation of $h(x)$, the kernel function must be stated:

$$K(x, z) = \langle h(x), h(z) \rangle$$

The kernel function computes the inner product in the transformed feature space without explicitly performing the transformation, which is referred to the kernel trick. For SVM optimization, the kernel must always be a symmetric positive (semi-definite) function.

Thus, the Lagrange (dual) function is:

$$L_D = \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i\alpha_j y_i y_j \langle h(x_i), h(z_j) \rangle$$

Then the decision boundary follows as:

$$\hat{f}(x) = \sum_{i=1}^{N} \hat{\alpha}_i y_i K(x, x_i) + \hat{\beta}_0$$

The following are some of the most commonly used kernels:

$$\begin{aligned} \text{d'th-Degree polynomial:} &\quad K(x, x') = (1 + \langle x, x' \rangle)^d \\ \text{Radial basis:} &\quad K(x, x') = exp(-\gamma \|x - x'\|^2) \\ \text{Neural network:} &\quad K(x, x') = tanh(\kappa_1 \langle x, x' \rangle + \kappa_2) \end{aligned}$$

As mentioned earlier, when using SVM, the parameter C must be estimated through cross-validation. If C is large, the model prioritizes classifying the training data correctly, minimizing training error but increasing the risk of overfitting. In contrast, a small C allows for some misclassification, encouraging better generalization to new data. Additionally, if a kernel like the radial basis function is used, then the parameter γ must also be chosen. Ultimately, testing different parameter combinations and tuning the model accordingly is essential to achieve the best classifier.

4 METHODOLOGY

In this section, a description of my methodologies in preparing the data and extracting features to build the SVM and RF models is given.

4.1 DATA PROCESSING

The data was downloaded from LOBSTER and for the price level 2 of 'SPDR S&P 500 ETF Trust' Ticker symbol (SPY). The SPY is an exchange-traded fund (ETF) that seeks to track the performance of the S&P 500 index. The specified period is from 01.04.2016 to 29.12.2023 which amounts to 2006 trading days and to an average of 251 trading days each year.

Figure 4 shows every step and order to process the collected data from Lobster.

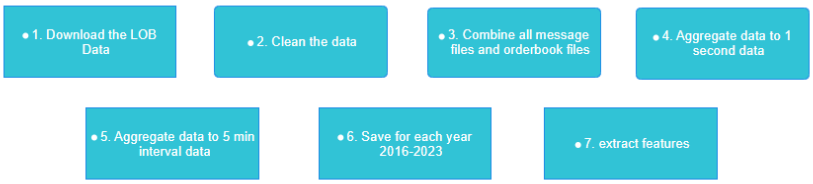

Figure 4: Steps in preparing data

In the first step, the high-frequency data for price level 2 LOB data for SPY for each year was downloaded. LOBSTER limits users to 100 GB of storage when requesting the data. Consequently, after retrieving the data for each year, we had to delete the data from their archive before requesting data for the following year. The data for each year took on average 10 hours to request and amounts to nearly 1 terabyte (TB) of data in total. For each year, the 'message files" and 'order book files' that were zero in size were removed. For the third step, we combined all message book and order book files to match the timestamp[1]. Given the vast amount of data, with highly granular timestamps from milliseconds to nanoseconds with so many millions of observations per trading day would be impractical to work with. Hence, the approach was to work with 1-seconds data instead which would be more practical and manageable. To obtain 1-second data, we took the

1 Files for combining are named the following `All Days 2016-2013.ipynb`

first observation for each second during the trading hours on Nasdaq Exchange that is from 9.30 AM to 4 PM. This amounts to

$$\text{seconds} = 6.5\,\text{hour} \cdot 60\,\text{minutes} \cdot 60\,\text{seconds} = 23.400$$

23400 observations for each feature per trading day.

Further, with 1-second frequency, some of the features may suffer from noise due to the highly granular data. To address this problem we can aggregate the data to 5-min interval which might help smooth out the noise and reveal informative patterns. Additionally, this will help reducing computational time and gives frequency for comparing. To aggregate 1-seconds to 5-min data the observations are reduced from 23.400 to 78 (intervals of 5-min) per day for each feature variable.

$$\frac{23.400\,\text{seconds}}{300\,\text{seconds}} = 78$$

4.2 FEATURES AND TECHNICAL INDICATORS

In the 1-second data frequency, we extracted the basic features from the LOB for n price level.

FEATURES DESCRIPTION	DETAILS
$F_1 = \{p_t^{ask}\}, \{V_t^{ask}\}, \{p_t^{bid}\}, \{V_t^{bid}\}_{=t}^{n}$	$n = 2$ Price Level
$F_2 = \text{Size}$	Number of shares
$F_3 = \{p_t^{ask}\} - \{p_t^{bid}\}, (\{p_t^{ask}\} + \{p_t^{bid}\}/2)_{=1}^{n}$	Spread, Mid-Price
$F_4 = \frac{1}{n}\sum_{=t}^{n}\{V_t^{ask}\}, \frac{1}{n}\sum_{=1}^{n}\{V_t^{bid}\}_{=t}^{n}$	Volume Means
$F_5 = \frac{\{V_t^{bid}\} - \{V_t^{ask}\}}{\{V_t^{bid}\} + \{V_t^{ask}\}}_{=t}^{n}$	Order Imbalance
$F_6 = \frac{\{p_t^{ask}\} \cdot \{V_t^{ask}\} + \{p_t^{bid}\} \cdot \{V_t^{bid}\}}{\{V_t^{bid}\} + \{V_t^{ask}\}}_{=t}^{n}$	VWAP

Table 1: 1-second features

For the 1-second data we have 12 basic features from the LOB. Additionally we have extracted 10 more features from $F_3 - F_6$ to our features set.

Due to data aggregation into 5-min interval one need to be cautious about how to choose the features to aggregate. One could for instance lose the ranges of how dynamic the prices of or the spread has been in those 5-min if one only take the means of all the bid and ask prices. To proper aggregate the 5-min data features, we have extracted the

first and last bid and ask prices, and highest and lowest prices value. Table 4 shows the structure of these features.

4.2.1 *Technical Indicators*

These [19], [17], and [23] found improvement and support of technical analysis in future price prediction, I will include some of the most used technical indicators used in the literature.

1. Simple Moving Average (SMA)[2] is widely used to identify trends.
2. Exponential Moving Average (EMA) is similar to SMA but puts more weight on recent prices. Fast at identifying trends in the short term.
3. The Relative Strength Index (RSI) is a momentum oscillator that measures the speed and change of price movements. When the RSI is above 70, it indicates the stock is overbought and when RSI is below 30, it indicates the stock is oversold.
4. Moving Average Convergence Diverence (MACD)[3] is a trend-following momentum indicator, where the signal line is typically a 9-period EMA in between a short term and long-term EMA. This line suggest whether or not the stock is has an upward trend or downward trend and triggers a signal to trade.
5. Bollinger Bands consists of a middle band with an upper and lower band that is typically two standard deviations away from the middle band. When the prices reaches or exceeds the upper band, the stocks is overbought and oversold when the stock is below the lower band.
6. Average True Range (ATR) measures the market volatility by calculating the average range between high and low prices over that specified period.

4.3 TRAIN, TEST, AND CROSS VALIDATION

In section 3.1. we talked about the goal of supervised learning is to learn a model from labeled training data that can make accurate predictions on unseen data. However, we cannot be sure that the model is reliable without evaluating its performance. To build such a model that we can rely on to make accurate out-of-sample predictions, it is crucial to split the available data into separate subsets for training and testing. This allows us to evaluate and validate how the model

2 Appendix.A2

3 I used the standard period of 9,12,26 for the 5-min and shorter periods of 6,8,20 for the 1-sec.

performs on unseen data during this testing process. Additionally, it is important to set aside an additional portion of the data available that the model has not seen during both training and testing. This portion of data will be used to test out-of-sample testing, where the performance will be evaluated and provides a measure of its predictive power.

One way to split the data into subsets of training and testing set could be the as proposed by Figure 5.

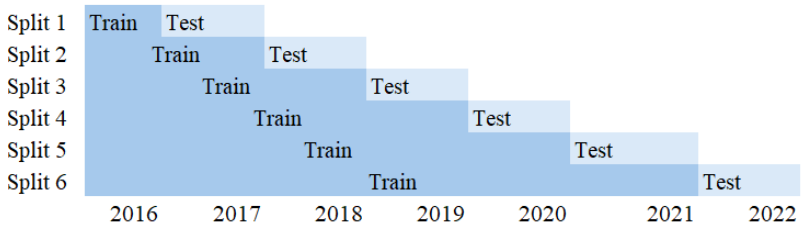

Figure 5: Rolling Window Cross Validation

Figure 5 shows the procedure of how our data is split for training and testing. We are training and validating from the years 2016-2022 and finally testing out-of-sample in 2023. The scheme is to train the full year trading data to then validating for the following in year. The steps are explained below.

- Train in 2016, validate in 2017.
- Train in 2016 and 2017, validate in 2018.
- Train in 2016, 2017, and 2018, validate in 2019.
- ...
- Train the data in 2016-2021, validate in 2022.

This method is referred to as Rolling Window Cross-Validation. In each split we are expanding the window of training set with one additional year to then validating the model the following year. The model with the best performance during this process will be used to perform the out-of-sample test for the reserved data in 2023.
This method of cross-validation to evaluate our models performance has both strength and weaknesses that we need to be wary about. When we split our subset of training and testing for the window size of one year, the model captures the longer seasonal trends and patterns, but the model might lose some insights in the short term. Conversely, if the window size is small we may not capture longer seasonal trends. In addition, this methods is also prone to overfitting and

skewed towards older training data. If say, market conditions in later years, 2021, is very different than the market conditions in 2016 and 2017 then the patterns the model has learned might fail to capture and adapt well to new market conditions. This is a problem where the model carries the importance of older training data to remain relevant in the future. This will in turn result in less accurate prediction and worse model performance for the new years in the validation set. Moreover, to cross-validate high-frequency data, the model has to be estimated and evaluated multiple times which is quite computational expensive and time consuming. With all the problems mentioned one might wonder why we still cross-validate using this method. Other approaches, for instance k-fold cross-validation, or stratified $k-$fold cross-validation would be an alternative. These approaches splits the subsets of data randomly into folds, trains the data $k-1$ folds, and validates the performance of the model on the remaining fold. This means after the full validation every fold has at least been validating the training data once. The problem with this approach is, it does not respect the order of time in time series data, resulting in future data can be shuffled and trained in the earlier years. Thus, the model performance is not reliable even when obtaining a high accuracy measure. These approaches would work well with data that are independent of each other, but in our case, we cross-validate using the rolling window cross-validation method that respects the order of time.

4.4 STANDARDIZATION OF THE DATA

Standardization of the data is a process to ensure that all input features contribute equally to the model performance. For some machine learning models standardization is not necessary but for others it is crucial. For the Random Forest model this process is not necessary due to its structure of features splitting at each decision node. These features will be averaged out from multiple decision trees. On the other hand, the SVM relies on standardization to give accurate model performance. The SVM relies on distance metrics to seperate hyperplanes, so if the data is not standardized, larger scale features might contribute more to the model performance than others even though these features are not important at all for the model performance.

4.5 TUNING THE MODEL

Tuning a machine learning model refers to the process to of finding the optimal model structure to get the best model performance. That is, we need to find the models' hyperparameters so the algorithm performs the best out-of-sample. In the case of the Random Forest algorithm, tuning the hyperparameters is to search for the optimal numbers of decision trees in the forest, the depth of trees etc.

A higher number of trees is generally better for the performance but at the expense of increased computational cost as we mentioned in section 3.2. A big advantage of this algorithmm is that, the ensemble method is robust to noise from the individual trees.
For the hyperparameters of the SVM Radial Basis Function (RBF) kernel we need to tune the regularization parameter C and the parameter γ. To tune C refers to controlling the trade-off between maximizing the margin and minimizing the classification errors. A higher C prioritizes to classify the training data correctly but runs the risk of overfitting, while a lower C allows for more classification errors on the training data but generalizes better on to new data, making it less prone to overfitting. The hyperparameter gamma, γ, defines the influence each data points have on the decision boundary. A higher γ has greater influence on data points, capturing more details in the data with a tight boundary decision, and runs the risk of overfitting. While a lower gamma has less influence on each data point and less tight boundary decision, but might not capture the important patterns in the model.

4.6 SCORING METRICS

Performance of the model is typically measured by calculating these scoring metrics - accuracy, recall, precision, and F1 score.
The accuracy measure is defined as

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

where TP represent true postives and TN is true negatives. FP and FN represent false positives and false negatives, respectively. The accuracy score metric measures the correct classification of mid-price going up or down. However, this measure only works appropriate on a dataset that is somewhat balanced otherwise it will be a misleading measure. In this case the two classes of up and down movement must be somewhat balanced. For instance, in an imbalanced dataset, where the up movements are 80 pct and only 20 pct movements are down, the accuracy can be misleading. A high accuracy in this case is just because there are more up movements than down movements, and the measure is unreliable. But, this measure works well for our 5-min data as it is balanced with 50.37 % up and 49.77 % down. Our 1-sec data is slightly less balanced, with 42.49 % and 57.51 % down. Overall our data seems balanced for us to use this measure.
We will primarily focus on the accuracy measure to evaluate the performance of our model. However, other measures such as precision, recall, and F1 score can also be important. Below, these measures are briefly defined.

Precision measures the proportion of mid-price upward movement that is predicted which turn out to be correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall measures the actual mid-price upward movement that is also classified as upward movement by the model.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The F1 score measures the balance between Precision and Recall, also referred to as the harmonic mean of precision and recall.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

A good model performance aims to achieve a relatively high score of all the measures mentioned.

5 EMPIRICAL ANALYSIS AND EVALUATION

We briefly restate the objective of this thesis and summarize our approach and methodology so far. The goal was to predict mid-price movements using high-frequency LOB data. Our aim was to evaluate whether a trading strategy based on these predictive models could outperform a simple buy-and-hold strategy, thereby challenging the EMH.

To do so, we have retrieved the SPY data from LOBSTER and combined the raw data into two dataset with different frequencies, specifically 5-min and 1-second interval LOB data from 2016-2023. The methodology includes feature extraction and incorporation of some widely used technical indicators in the literature. To classify the mid-price movements, we applied the Support Vector Machine and Random Forest classifiers. Using a rolling window cross-validation from 2016-2022, we identified the most effective model based on accuracy and overall score metrics. The best performing model was then used to test out-of-sample for the data of 2023. To ensure robustness we evaluated the performance on 2023 and then simulated our trading strategy. In the following sections, we will describe and present our validated models and results.

5.1 CROSS-VALIDATION AND FEATURE IMPORTANCE

After processing the data as outlined in the methodology section 4.1, we applied the basic features together with additionally defined extracted features from the LOB data. These features were used for both frequency intervals to capture relevant market dynamics. This approach ensures the model's improved ability across different levels of granularity. Applying the above and the technical indicators outlined earlier, we trained and validated with a rolling window cross-validation through 2016-2022 for both frequencies (5-min and 1-sec[1]). This is done for the RF classifier with the hyperparameter gridsearch[2].
As mentioned in the section of the Random Forest, the larger number of trees generally improves the model performance but at the expense of increased computational time. For this trade-off, we have chosen to reduce the number of tress and other parameters to reduce the

1 Reults in appendix A.5.3
2 See appendix A.3

computational time. In our cross-validation with 324 hyperparameter combinations and 1944 models to evaluate in total, this process requires about 12 hours of computation time on an Intel i5 processor with 6 cores and 12 threads. The results from grid search hyperparameter tuning, obtained from cross-validation, are presented Table 2.

5.1.1 *Random Forest 5-min cross-validation*

2016-2022	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1	49.70	49.54	49.70	49.11
Split 2	49.62	49.37	49.62	41.71
Split 3*	50.16	50.23	50.16	47.46
Split 4	49.72	49.27	49.72	44.88
Split 5	48.90	23.91	48.90	31.11
Split 6	50.08	49.92	50.08	45.61

Table 2: Cross-Validation resulsts in percentage % (5-min).
Fitting 6 folds for each 324 candidates, total 1944 models.
*: Best model
Total time taken for estimation : 42.222,41 seconds ~ 12 hours

From the table we see that split 3 is the best performing model in terms of accuracy and overall score metrics. An accuracy and the other score metrics about 50 % indicates a model that is no better than random guessing, like tossing a coin. An indication of a poor model for capturing the patterns in the data. Possible reasons could be the smaller number of trees and reduced parameters or even the need for more or different features.
Although, the optimal model is used to test out-of-sample for the 2023 data. We are using the best parameters[3] from the model of split 3 to do so in section 5.2.

For the case of SVM, we applied the same technical indicators and features for cross-validation. The results are somewhat similar to the Random Forest with an overall score metrics about 50 & which is also a poor performing model. The results are presented in table 3. Cross-validating using the Support Vector Machine Radial Basis Function (RBF) kernel is computational intensive. With "only" 12 hyperparameter combinations and 72 models total computational time is nearly 10 hours. We proceed with the best performed model of split 4 for out-of-sample testing.

3 `Random Forest 5.min (Right) Estimation.ipynb`

5.1.2 *SVM 5-min cross-validation*

2016-2022	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1	50.67	25.67	50.67	34.08
Split 2	50.12	25.12	50.12	33.47
Split 3	49.95	24.95	49.95	33.28
Split 4*	50.75	50.74	50.75	48.48
Split 5	49.03	50.75	49.03	34.53
Split 6	50.32	50.91	50.32	35.93

Table 3: Cross-Validation resulsts in percentage % (5-min).
Fitting 6 folds for each 12 candidates, total 72 models.
*: Best model
Total time taken for estimation : 34.989,80 seconds ~ 10 hours

We have also trained our model with the 1-sec dataset for comparison measure, but only for the Random Forest due to SVM is quite computational intensive. The results are presented in the appendix. The best performed mode, has an accuracy of 71.5 % and other metrics ranging from 60-70 percent. In contrast to the SVM and RF for the 5-min frequency, this model performs well and might be better in capturing the patterns in the LOB data.

5.1.3 *Random Forest Feature Importance*

Features importance provide models insights of which features are most important in making predictions. Each time a feature is used to split a node in the tree, it reduces the gini impurity or maximizing information gain by separating classes more effectively. Features in our dataset that reduce the impurity significantly are considered most important as mentioned earlier. The importance of each feature is the sum of gini impurity reduction that contributes across all splits in all trees in the forest. The sum is weighted and normalized to 1. Feature importance in Random Forest models is inherent from the cross-validation. Figure 6 below are output from features importance scores from hyperparameter tuning and cross-validating using 1-sec dataset from 2016-2022.

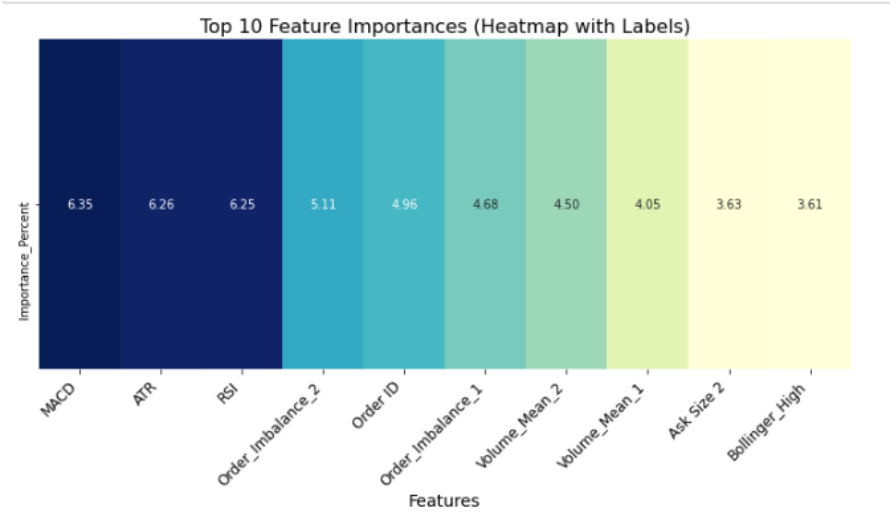

Figure 6: RF top 10 feature importance

We see that the features of MACD, RSI, ATR is top three importance when performing a split. The percentage of about 6 percentage indicates a moderate importance. But overall the top 10 features that contributes the most are about of 50 % in the overall scores of features importance. In the 5-min Random Forest features importance the top two features MACD and RSI and have highly importance, that is, ranging from 10-17 percent.

5.2 OUT-OF-SAMPLE RESULTS AND TRADING STRATEGY

5.2.1 *Out-of-sample 2023*

As mentioned in the previous section, we use the best performing models in terms of accuracy and the other score metrics to evaluate out-of-sample performance of 2023 data. The approach is to proceed in two step. We are validating the out-of-sample data with the same features and technical indicators for both frequency dataset and predictive models of the SVM and RF. After this step, we then simulate our trading strategies given the best model here. Instead of proceed in one step, that is, simulate out-of-sample immediately using the best model from the cross-validation. We are evaluating once more on the unseen data. This is due to our poor model performance for the SVM and RF for the frequency of 5-min. Also, this approach will give us an indication of the models ability of generalized performance.

The results of evaluation for the 2023 the best model is still the split 3. But with an minimal increased accuracy of 50.97 % in the RF 5-min models. For the SVM we get a different performance model. The best model is split 5 instead of split 4 with an accuracy score of slightly worse than previous, 49.73 %. Lastly, for the 1-sec we get an accuracy

of 55.81 %, but still the first split as the best model. That is, this model accuracy has reduced from 71.5 % to 55.81 %.

5.2.2 *Trading Strategy*

We implement our trading strategy given these best model performances for both frequencies. This strategy is then compared to the benchmark of the buy-and-hold. The strategy has the following rules:

- We start by initial capital of 10.000 available for trading.
- We track the shares held at each time.
- When the models predicts the next mid-price as an upward movement we use all capital and buy shares.
- When we model predicts the next mid-price as an down movement we sell all shares held.
- At each event the trading history is tracked.
- At the end of the period (year) if we have any remaining shares, we sell all of them.
- The profit and loss is compared to initial capital of 10.000.

The buy-and-hold strategy is to buy shares at the start of the year with available capital of 10.000 at the price at the time. Holding throughout the year and sell all shares at the of the year.

We implement the simulation with and without transaction costs. To incorporate transaction costs for each trade will give us a more realistic real-world trading scenario. We incorporated the transactions costs of 0.1 % per trade, that is, including brokerage fees, slippage etc.

STRATEGY	(RF5MIN)	(SVM5MIN)	(RF1-SEC)
W/O	12.430,4	10.972,6	10.054,1
W	12.257,5	1.435,8	1.302,1
B&H	12.400,5	12.534,8	9.874,3

Table 4: Profit and loss comparison of across models

From the table we see the profit and loss for all three models. In the first row, the strategy is without transaction costs, second row is with transaction costs, and the final row is the buy and hold for comparison[4]. Both Random Forest model outperform the buy-and-hold strategy while the SVM does not.

4 Including transaction costs for buy and hold does not change the results and conclusion.

5.3 THE EFFICIENT MARKET HYPOTHESIS

The Efficient Market Hypothesis (EMH) stresses that, at any instant time, the price of a security equals it fundamental value. According to Fama (1970) [1] all publicly available information is immediately incorporated into stock prices and that no excess returns can be achieved through any trading strategy. Because, any potential profit opportunities will be captured by the market and quickly be eroded. Historical price patterns cannot be predicted to perform strategies that outperform the market in the long-term. In the literature, challenges to the EMH has been showed. An example is Shiller (1981)[26] shows there has been excess volatility and the stock prices seems too high in comparison to what is stated by the EMH. Another is De Bondt and Thaler (1985) [27] that used patterns of price-earning ratios from top and bottom 35 firms from 1933-1980, and evidently showed that firms with lower price-earnings ratio in this period have subsequently higher stock prices in the following years. Conversely, the top firms with higher price-earnings ratios have lower stock prices the subsequent years for the test period of 3-5 years. These evident show that historical data and patterns can be used to challenge the EMH.

Our contribution with this thesis is to use the new settings of modern dynamics of electronic trading and HFT Limit Order Book data to identify and exploit patterns in historical data, thus challenging the EMH.

The EMH implies that short-term price discrepancies can exists due to various market factors e.g. different information processing time. However, EMH suggets, that on average the market will correct itself over time. Rather than focusing on short-term patterns, we have used data for eight years, from 2016-2023, which we define as the long-term.

Our results from the trading strategies, without incorporating transactions costs, indicate that patterns in historical data combined with machine learning algorithms may give predictive insights into future stock price movements. Although these findings are based on the results without transactions costs and not real-world scenario, they highlight that with optimized model training and feature extraction, the modern era of electronic trading and information could impact the traditional assumptions of the EMH.

6 DISCUSSION AND CONCLUSION

6.1 DISCUSSION AND LIMITATION

In this section, we will evaluate our findings and discuss the methodology taken.
In the literature review, we mentioned several studies where authors used HFT Limit Order Book data and machine learning algorithms to conduct their mid-price prediction. Although, our findings indicate predictive patterns may exists in the data, challenging the efficiency of the EMH, these results should be interpreted with skepticism.
Our approach could be significantly improved with increased computational time. For instance, increasing the number of trees in the Random Forest model to 500 or 1000, and expanding other model parameters to achieve the better model optimization. In the case of SVM, incorporating feature importance could also improve the performance. While the SVM does not directly gives us feature importance, we could for example, include recursive feature elimination. That is, eliminating features that do not contribute to the model performance when cross-validating. This could improve our model further. This approach was not done due to the trade-off of computational time. Another approach was cleaning the data extensively, by excluding some outliers etc.
Even though, we are novice in this area and our findings should be seen with skepticism, the purpose of this thesis was solid. This approach could be interesting by using other machine learning algorithm. Further, this could include more stocks for a more rigorous analysis.

6.2 CONCLUSION

As outlined in the introduction, the goal was to predict the movements of future mid-prices from Limit Order Book data by applying two machine learning algorithms well-known in the literature. The Random Forest and The Support Vector Machine algorithms. Additionally, the aim was to develop a trading strategy based on these predictive models to evaluate whether it could outperform the market over the long-term, thereby challenging the efficiency of the Efficient Market Hypothesis. Our findings did find predictive patterns in historical data that the strategy formulated outperformed the benchmark strategy of the EMH without transaction costs. Although, incor-

porating transactions costs our model did no find results in challenging the efficiency of the EMH.

APPENDIX

A.1 TRADING HALTS (MESSAGE BOOK)

trading halts.

When trading halts, a message of type '7' is written into the 'message' file. The corresponding price and trade direction are set to '-1' and all other properties are set to '0'. Should the resume of quoting be indicated by an additional message in NASDAQ's Historical TotalView-ITCH files, another message of type '7' with price '0' is added to the 'message' file. Again, the trade direction is set to '-1' and all other fields are set to '0'. When trading resumes a message of type '7' and price '1' (Trade direction '-1' and all other entries '0') is written to the 'message' file. For messages of type '7', the corresponding order book rows contain a duplication of the preceding order book state.

The figure below illustrates the trading halt messages as they appear in the message file.

Time (sec)	Event Type	Order ID	Size	Price	Direction
⋮	⋮	⋮	⋮	⋮	⋮
34713.685155243	7	0	0	-1	-1
⋮	⋮	⋮	⋮	⋮	⋮
34719.193632201	7	0	0	0	-1
⋮	⋮	⋮	⋮	⋮	⋮
34801.333638703	7	0	0	1	-1
⋮	⋮	⋮	⋮	⋮	⋮

To keep the output structure as clean as possible, the reason for the trading halt is not included in the output.

Figure 7

A.2 FEATURES AND TECHNICAL INDICATORS (TI)

FEATURES DESCRIPTION	DETAILS
$F_1 = \{p_t^{amin}\}, \{p_t^{amax}\}, \{p_t^{afirst}\}, \{p_t^{alast}\}_{=t}^{n}$	$n = 2$ Price Level
$F_2 = \{p_t^{bmin}\}, \{p_t^{bmax}\}, \{p_t^{bfirst}\}, \{p_t^{blast}\}_{=t}^{n}$	$n = 2$ Price Level
$F_3 = \{p_t^{amin}\} - \{p_t^{bmin}\}, \{p_t^{amax}\} - \{p_t^{bmax}\}_{=t}^{n}$	Spread
$F_4 = \{p_t^{afirst}\} - \{p_t^{bfirst}\}, \{p_t^{alast}\} - \{p_t^{blast}\}_{=t}^{n}$	Spread
$F_5 = (\{p_t^{amin}\} + \{p_t^{bmin}\}/2), (\{p_t^{amax}\} + \{p_t^{bmax}\}/2)_{=t}^{n}$	Mid-Price
$F_6 = (\{p_t^{afirst}\} + \{p_t^{bfirst}\}/2), (\{p_t^{alast}\} + \{p_t^{blast}\}/2)_{=t}^{n}$	Mid-Price
⋮	⋮
⋮	⋮
⋮	⋮
$F_{60} =$	VWAP

Table 5: 5-min features 60 features

1. Simple moving Average (SMA)

$$SMA_t = \frac{\sum_{i=0}^{n-1} P_{t-i}}{n}$$

where P_{t-i} is the price and i is between 0 to $n-1$ taking the sum of prices and average it over n periods.

2. Exponential Moving Average (EMA)

$$EMA_t = (V_t \cdot \alpha) + (EMA_{t-1} \cdot (1 - \alpha))$$

where V_t is the value at time t, $\alpha = \frac{2}{n+1}$ is the smoothing factor and n is the number of periods.

3. 14-period Relative Strength Index (RSI)

$$RSI = 100 - \frac{100}{1 + RS}$$

where $RS = \frac{\text{average gain}}{\text{average loss}}$ is the relative strength over a period $n = 14$.

4. The Moving Average Convergence Divergence (MACD)

$$\text{MACD line} = EMA_{12} - EMA_{26}$$

where EMA_{12} is of 12 periods and EMA_{26} is of 26 periods. Also, the signal line

$$\text{signal line} = EMA_9 \cdot (\text{MACD line})$$

where EMA_9 is of 9 periods.

5. Bollinger Bands (BB)

$$\begin{aligned} \text{middle band} &= SMA(P, n) \\ \text{upper band} &= \text{middle band} + (k + SD(P, n)) \\ \text{lower band} &= \text{middle band} - (k + SD(P, n)) \end{aligned}$$

where SMA is the simple moving average, P is the price and n is the number of periods. Note also that k is the number of standard deviations and $SD(P, n)$ is the standard deviation of price over n periods.

6. Average True Range (ATR)

$$ATR = \frac{1}{n}\sum_{i=1}^{n} TR_i$$

where the true range $TR = max(high, close_{prev}) - min(high, close_{prev})$ and $ATR_t = \frac{ATR_{t-1} \cdot (n-1) + TR_t}{n}$ for n periods.

A.3 PYTHON 3.8.8 PACKAGES AND GRIDSEARCH

Sklearn.ensemble - RandomForestClassifier.
Sklearn.model_selection - GridSearchCV, Timeseriessplit.
Pandas.
Numpy.
sklearn.metrics.
sklearn.preprocessing - StandardScaler.
sklearn.svm - svc.
matplot.pyplot.
Seaborn.

```
RF5minparam_grid = {
    'n_estimators': [100, 150, 200],
    'criterion': ['gini'],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['auto', 'sqrt'],
    'bootstrap': [True, False]
```

```
RF1secparam_grid = {
    'n_estimators': [50, 100, 150],
    'criterion': ['gini'],
    'max_depth': [10, 20, 25],
    'min_samples_split': [2, 5, 8],
    'min_samples_leaf': [1, 2, 5],
    'max_features': ['auto', 'sqrt'],
    'bootstrap': [True, False]

SVM5minparam_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['rbf'],
    'gamma': ['scale', 0.1, 1, 10]
```

A.4 CROSS-VALIDATION RESULTS

A.4.1 *Random Forest 1-sec cross-validation*

2016-2022	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1*	71.50	65.43	71.50	59.98
Split 2	64.49	41.59	64.49	50.57
Split 3	56.56	54.11	56.56	53.01
Split 4	60.38	57.09	60.38	57.30
Split 5	49.13	51.79	49.13	48.90
Split 6	62.97	57.31	62.97	52.57

Table 6: Cross-Validation resulsts in percentage % (1-sec).
Fitting 6 folds for each 162 candidates, total 972 models.
*: Best model
Total time taken for estimation : 838,30 seconds ~ 24 min

A.5 OUT-OF-SAMPLE TESTING 2023 RESULTS

A.5.1 *Random Forest 5-min out-of-sample results*

2023	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1	49.70	49.54	49.70	34.11
Split 2	48.97	49.16	48.97	35.85
Split 3*	50.97	48.61	50.97	34.47
Split 4	49.03	50.47	49.03	32.39
Split 5	49.01	24.04	49.03	32.26
Split 6	49.01	24.02	49.01	32.24

Table 7: RF out-of-sample test results in percentage % (5-min).
*: The best model to build our trading strategy upon.

A.5.2 *SVM 5-min out-of-sample results*

2023	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1	49.69	50.94	49.69	41.92
Split 2	49.67	49.80	49.67	49.50
Split 3	48.83	49.11	48.83	45.38
Split 4	49.28	49.58	49.28	47.86
Split 5*	49.73	49.94	49.73	49.28
Split 6	49.60	49.54	49.60	49.52

Table 8: SVM out-of-sample test results in percentage % (5-min).
*: The best model to build our trading strategy upon.

A.5.3 *Random Forest 1-sec out-of-sample results*

2023	ACCURACY	PRECISION	RECALL	F1 SCORE
Split 1	49.69	50.94	49.69	41.92
Split 2	49.67	49.80	49.67	49.50
Split 3	48.83	49.11	48.83	45.38
Split 4	49.28	49.58	49.28	47.86
Split 5*	49.73	49.94	49.73	49.28
Split 6	49.60	49.54	49.60	49.52

Table 9: RF out-of-sample test results in percentage % (1-sec).
*: The best model to build our trading strategy upon.

BIBLIOGRAPHY

[1] Eugene F. Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work." In: *The Journal of Finance* Volume 25.Issue 2 (1970), pp. 383–417 (cit. on pp. 1, 28).

[2] Addy et al. "Algorithmic Trading and AI: A review of strategies and market impact." In: *World Journal of Advanced Engineering Technology and Sciences* Volume 11.Issue (2014), pp. 258–267 (cit. on p. 2).

[3] Ehsan Hoseinzade, Saman Haratizadeh. "CNNpred: CNN-based stock market prediction using a diverse set of variables." In: *Expert Systems with Applications* Volume 129 (2019), pp. 273–285 (cit. on p. 2).

[4] Bruno Miranda Henrique, Vinicius Amorim Sobreiro, Herbert Kimura. "Stock Price Prediction using Support Vector Regression on Daily and up to the minute prices." In: *The Journal of Finance and Data Science* Volume4.Issue 3 (2019), pp. 183–201 (cit. on p. 2).

[5] Zihao Zhang, Stefan Zohren, Stephen Roberts. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." In: *IEEE Transactions on Signal Processing* Volume 67.Issue 11 (2019), pp. 3001–3012 (cit. on p. 2).

[6] Justin Sirignano, Rama Cont. "Universal features of price formation in financial markets: perspectives rom deep learning." In: *Quantitative Finance* Volume 19.Issue 9 (2019), pp. 1449–1459 (cit. on p. 2).

[7] Xuekui Zhang et. al. "Novel modelling strategies for high frequency trading data." In: *Journal of Financial Innovation* Volume 9.Issue 9 (2022), pp. 1–25 (cit. on p. 2).

[8] Jan Grudniewicz, Robert Slepazuk. "Application of machine learning in algorithm investment strategies on glabal stock markets." In: *Journal of Research in International Business Finance* Volume 66.Issued (2023) (cit. on p. 2).

[9] James Han et al. "Machine learning techniques for price change forecast using th e limit order book data." In: Volume.Issue (2015) (cit. on p. 2).

[10] Nouise et al. "Machine learning fore forecasting mid price movemetn using LOB data." In: *Journal of Computational Engineering, Finance, and Science* Volume.Issue (2019) (cit. on p. 2).

[11] To-Han Chang. "Stock Price Prediction Absed on Data Mining Combination Model." In: *Journal of Global Information Management* Volume 30.Issue 7 (2021) (cit. on p. 2).

[12] Adamantios Ntakaris, Martin Magris, Juho Kannianen, Moncef Gabbouj, Alexandros Iosifidis. "Benchmark Dataset for mid-price forecasting of limit order book data with machine learning methods." In: *Journal of Forecasting* Volume 37.Issue 8 (2018), pp. 852–866 (cit. on p. 2).

[13] Alec N. Kercheval, Yuan Zhang. "Modelling high-frequency limit order book dynamics with support vector machines." In: *Quantative Finance* Volume 15.Issue 8 (2015), pp. 1315–1329 (cit. on pp. 2, 3).

[14] Manveer Kaur Manget, Erhard Reshcenhofer, Thomas Stark and Christian Zwatz. "High-Frequency Trading with Machine Learning Algorithms and Limit Order Book Data." In: *Data Science in Finance And Economics* Volume 2.Issue 4 (2022), pp. 437–463 (cit. on pp. 2, 3).

[15] Hiransha et. al. "NSE Stock Market Prediction Using Deep-Learning Models." In: *Procedia Computer Science* Volume 132.Issue (2018), pp. 1351–1362 (cit. on p. 3).

[16] Ebrahim Abbasi et al. "Performance Evaluation of the Technical Analysis Indicators in Comparison with the Buy and Hold Strategy in Tehran Stock Exchange Indices." In: *Advances in Mathematical Finance and Applications* Volume 5.Issue 3 (2020) (cit. on p. 3).

[17] Michael Frömmel, Kevin Lampeart. "Does frequency matter for intraday technical trading." In: *Journal of Finance Research Letters* Volume 18.Issue (2015), pp. 177–183 (cit. on pp. 3, 18).

[18] Xiaoye Jin. "Evaluating the predictive power of intraday trading in China's crude oil market." In: *Journal of Forecasting* Volume 41.Issue 7 (2022), pp. 1416–1432 (cit. on p. 3).

[19] Gil Cohen, Elinor Cabiri. "Can technical oscillators outperform the buy and hold strategy." In: *Applied Economics* Volume 47.Issue 30 (2015), pp. 3189–3197 (cit. on pp. 3, 18).

[20] Robert F. Engle. "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation." In: *Econometrica* Volume 50.Issue 4 (1982), pp. 987–1007 (cit. on p. 3).

[21] Eugene F. Fama, Kenneth R French. "Common risk factors in the returns on stocks and bonds." In: *Journal of Financial Economics* Volume 33.Issue 1 (1993), pp. 3–56 (cit. on p. 3).

[22] Shihao Gu, Bryan T. Kelly, Dacheng Xiu. "Empirical Asset Pricing Via Machine Learning." In: *The Review of Financial Studies* Volume 33.Issue 5 (2018), pp. 2223–2773 (cit. on p. 3).

[23] Campisi et al. "A comparison of machine learning methods for predicting the direction of the US stock markket on the basis of volatility." In: *International Journal of Forecasting* Volume 40.Issue 3 (2024), pp. 869–880 (cit. on pp. 3, 18).

[24] Raschka and Mirjalili. *Pythong Machine Learning*. Packt Publishing, 2017 (cit. on pp. 7, 9).

[25] Jerome Friedman Trevor Hastie Robert Tibshirani. *The Elements of Statistical Learning*. Springer, 2009 (cit. on p. 11).

[26] Robert j. Shiller. "Do Stock Prices Move Too much to be justified by subsequent changes in dividends?" In: *The American Economic Review* Volume 71.Issue 3 (1981), pp. 421–436 (cit. on p. 28).

[27] De Bondt, Richard Thaler. "Does the stock market overreact." In: *The Journal of Finance* Volume 40.Issue 3 (1985), pp. 793–805 (cit. on p. 28).